Pendientes: Menina Coiffure

Resumen

Este documento lista lo que conviene corregir tras probar el frontend de Reservas contra la rama `frontThiago` (commit `f528739`), ordenado por prioridad.

Cada pendiente indica cómo reproducirlo, qué impacto tiene, un arreglo sugerido y cómo se verificó. La verificación se marca así:

- **Ejecutado**: se reprodujo contra el backend real.
- **Leído en el código**: confirmado en el repositorio, sin ejecutarlo.
- **Sin verificar**: deducido o pendiente de comprobar; conviene reproducirlo antes de arreglarlo.

Son 15 pendientes: 2 de prioridad P1, 5 de P2 y 8 de P3. Los 15 se verificaron leyendo el código; ninguno se ha reproducido todavía contra el backend real, salvo el comportamiento descrito en el punto 14. Se sugiere atender primero los P1.

Prioridad P1: bloquean funciones de la demo

Dos pendientes impiden mostrar funciones clave de la aplicación: crear reservas desde la interfaz y cancelarlas a tiempo.

1. Falta el endpoint `/api/clientes`

- **Qué pasa:** `routes/api.php` no define rutas de clientes y no existe un `ClienteController`; solo existe el modelo `Cliente`.
- **Cómo reproducir:** `GET /api/clientes` debe responder 404.
- **Impacto:** el frontend no puede listar clientes, así que no se puede construir "Nueva reserva" ni la pantalla de Clientes.
- **Arreglo sugerido:** `Route::apiResource('clientes', ClienteController::class)` con, al menos, `index` (con búsqueda por nombre) y `store`.
- **Verificación:** leído en el código.

2. La zona horaria de la aplicación es UTC

- **Qué pasa:** `config/app.php` fija `'timezone' => 'UTC'`. Las horas de las reservas se guardan como hora local de Asunción (UTC-3), pero `Carbon::parse("{fecha} {hora_inicio}")` las interpreta como UTC. El sistema cree que la cita empieza 3 horas antes de lo real.

- **Cómo reproducir (a):** crear una reserva para hoy, unas 3 horas adelante, y cancelarla con `DELETE /api/reservas/{id}`. Debe responder 422 con "No se puede cancelar una reserva cuya fecha y hora de atención ya ha transcurrido".
- **Cómo reproducir (b):** después de las 21:00 hora de Asunción, `POST /api/reservas` con la fecha de hoy debe responder 422 con "La fecha de la reserva no puede ser anterior al día de hoy", porque `after_or_equal:today` usa la fecha UTC, que ya es la del día siguiente.
- **Impacto:** la cancelación queda bloqueada durante las 3 horas previas a la cita, y entre las 21:00 y las 24:00 no se pueden crear reservas para el día en curso. El frontend decide con la hora del navegador, así que ofrece acciones que el backend rechaza.
- **Arreglo sugerido:** `'timezone' => env('APP_TIMEZONE', 'America/Asuncion')` en `config/app.php`, `APP_TIMEZONE=America/Asuncion` en `.env` y `php artisan config:clear`. Efecto lateral: los `created_at` y `updated_at` nuevos se guardarán en hora local.
- **Verificación:** leído en el código; no se ha ejecutado. El caso (b) se puede comprobar entre las 21:00 y las 24:00 hora de Asunción.

Prioridad P2: errores conocidos

Cinco errores de datos o de validación que no impiden la demo, pero dejan estados inconsistentes o devuelven errores 500.

3. Borrar una categoría con cascada puede dar un error 500

- **Qué pasa:** `CategoriaServicioController@destroy` hace `delete()` sobre la categoría, y `servicios.categoria_id` borra en cascada sus servicios. Pero `detalle_reserva.servicio_id` referencia `servicios` sin cascada: si algún servicio de la categoría ya está en una reserva, la base de datos rechaza el borrado y la excepción no se captura (500). Si ningún servicio está en reservas, el borrado procede y elimina servicios y precios sin aviso.
- **Cómo reproducir:** con una reserva que incluya un servicio de la categoría, `DELETE /api/categorias-servicio/{id}`.
- **Impacto:** error 500 sin mensaje útil, o pérdida silenciosa de servicios y precios.
- **Arreglo sugerido:** responder 422 si la categoría tiene servicios, o desactivarla en lugar de borrarla, como ya se hace con servicios y personal.
- **Verificación:** leído en el código; el 500 está pendiente de reproducir con una reserva real.

4. Cambiar el precio dos veces el mismo día deja un rango de fechas invertido

- **Qué pasa:** `ServicioController@cambiarPrecio` cierra el precio vigente con `vigente_hasta` igual a la fecha de cambio menos un día. Si ese precio había empezado el mismo día, queda con `vigente_desde` posterior a `vigente_hasta`.
- **Cómo reproducir:** `POST /api/servicios/{id}/cambiar-precio` dos veces el mismo día con precios distintos, y revisar la tabla `precios_servicio`.
- **Impacto:** queda una fila de historial con rango inválido (desde D, hasta D-1). El precio vigente se sigue calculando bien, pero el historial es inconsistente.
- **Arreglo sugerido:** si el precio vigente empieza en la fecha del cambio, actualizarlo en lugar de cerrarlo y crear otro, y validar que `vigente_desde` no sea anterior al inicio del precio vigente.
- **Verificación:** leído en el código.

5. Los turnos no validan sus horas ni los solapamientos

- **Qué pasa:** `PersonalController@guardarTurnos` valida el formato de hora y el día, pero no que `hora_fin` sea posterior a `hora_inicio` ni que las franjas de un mismo día no se solapen. Además, la regla `required` sobre `turnos` rechaza un arreglo vacío, por lo que no se puede dejar a alguien sin turnos.
- **Cómo reproducir:** `POST /api/personal/{id}/turnos` con `hora_inicio` `18:00` y `hora_fin` `09:00` debería guardarse sin error.
- **Impacto:** un turno invertido no cubre ningún rango y rechaza toda reserva de ese día. Al reservar se exige un solo turno que cubra la cita completa, así que una cita que cruza dos franjas contiguas (por ejemplo 09:00 a 11:00 y 11:00 a 13:00) se rechaza aunque el personal esté disponible.
- **Arreglo sugerido:** validar en cada franja que `hora_fin` sea posterior a `hora_inicio` (con `after` por elemento o una regla personalizada) y que no se solapen dentro del día; evaluar unir franjas contiguas al validar reservas.
- **Verificación:** leído en el código.

6. Desactivar personal no considera sus reservas futuras

- **Qué pasa:** `PersonalController@update` (con `activo: false`) y `destroy` solo cambian `activo`; no consultan reservas futuras. Además, `ReservaService@crearReserva` solo verifica que el personal exista, no que esté activo.
- **Cómo reproducir:** desactivar a un profesional con reservas futuras (`DELETE /api/personal/{id}`) y luego `POST /api/reservas` con ese `personal_id`. Ambas operaciones deberían tener éxito.

- **Impacto:** quedan reservas futuras asignadas a personal inactivo y se pueden crear nuevas.
- **Arreglo sugerido:** al desactivar, responder 422 o pedir reasignar las reservas pendientes o confirmadas futuras; al crear una reserva, validar que el personal esté activo.
- **Verificación:** leído en el código.

7. Diana no tiene usuario en el seeder

- **Qué pasa:** `SprintUnoDosSeeder` crea los usuarios `admin`, `recepcion` y `carla`, pero ninguno para Diana, aunque `diana@meninacoiffure.com` existe como personal.
- **Cómo reproducir:** ejecutar `php artisan db:seed` y revisar la tabla `users`, o intentar iniciar sesión con ese correo.
- **Impacto:** no se puede probar el rol profesional con Diana ni el comportamiento con varios profesionales.
- **Arreglo sugerido:** agregar un `User::updateOrCreate` para Diana con el rol profesional y su `personal_id`, igual que el de Carla.
- **Verificación:** leído en el código.

Prioridad P3: mejoras

Ocho mejoras que reducen reglas sin validar, datos faltantes y costo de rendimiento.

8. `PUT /api/reservas/{id}` no valida transiciones de estado

- **Qué pasa:** `ReservaService@actualizarReserva` solo bloquea el cambio si el estado actual es completada o cancelada. Permite cualquier otro salto, como pendiente a completada o confirmada a pendiente. Además, enviar `estado: cancelada` por PUT se salta la regla de `cancelarReserva` según la cual una cita ya transcurrida no se cancela.
- **Arreglo sugerido:** definir un mapa de transiciones permitidas y delegar la cancelación en `cancelarReserva`.
- **Verificación:** leído en el código.

9. `Reserva` no tiene relación `descuento`

- **Qué pasa:** la tabla `descuentos` y la llave `reservas.descuento_id` existen, pero no hay modelo `Descuento` ni relación en `Reserva`, así que el listado no puede traer el descuento. El frontend solo avisa que el subtotal no lo incluye.
- **Arreglo sugerido:** crear el modelo `Descuento`, la relación `descuento()` y cargarla en `index` y `show`; idealmente, devolver también el total con descuento.

- **Verificación:** leído en el código.

10. Las reservas pendientes ya pasadas no se pueden cancelar y no existe el estado "no asistió"

- **Qué pasa:** `cancelarReserva` rechaza toda cita cuya hora de inicio ya pasó, y los estados válidos son pendiente, confirmada, cancelada y completada. Una cita a la que nadie asistió queda pendiente para siempre; el frontend la marca como "Sin cerrar".
- **Arreglo sugerido:** agregar el estado `no_asistio` o permitir cerrar las citas ya pasadas.
- **Verificación:** leído en el código.

11. El listado de reservas no tiene filtros ni tope de `per_page`

- **Qué pasa:** `GET /api/reservas` solo acepta `page` y `per_page`, sin máximo. El frontend carga todas las páginas con `per_page=100` y filtra en el navegador, por lo que el costo de carga crece con todo el histórico. Es evidencia útil para el benchmark del Avance 4 (27 de noviembre).
- **Arreglo sugerido:** aceptar `fecha_desde`, `fecha_hasta`, `estado`, `personal_id` y `cliente_id`, y poner un tope a `per_page`.
- **Verificación:** leído en el código.

12. Los seeders no crean reservas

- **Qué pasa:** ni `AvanceDosSeeder` ni `SprintUnoDosSeeder` insertan reservas, así que hay que crearlas a mano para probar. Tampoco hay datos de volumen para el benchmark.
- **Arreglo sugerido:** un seeder con reservas de ejemplo y otro de volumen (miles de reservas) para medir RPS y latencia.
- **Verificación:** leído en el código.

13. No existe `/api/reportes`

- **Qué pasa:** `routes/api.php` no define rutas de reportes, y la pantalla de Reportes del frontend queda en "Pronto". Mientras tanto se podría calcular en el navegador desde `/api/reservas`, con el costo del punto 11.
- **Arreglo sugerido:** acordar con el frontend qué métricas se necesitan y exponerlas en endpoints agregados.
- **Verificación:** leído en el código.

14. Los servicios solo se pueden desactivar, no eliminar

- **Qué pasa:** `ServicioController@destroy` pone `activo = false`, y un servicio desactivado sigue apareciendo como inactivo en la pantalla de Servicios. Para quitar un servicio de prueba hoy hay que borrarlo en la base de datos. Allí, `precios_servicio` se borra en cascada pero `detalle_reserva` no, así que un servicio usado en reservas no se puede borrar.
- **Arreglo sugerido:** confirmar si es decisión de diseño; si no lo es, permitir borrar los servicios sin reservas.
- **Verificación:** leído en el código; el comportamiento de la pantalla se observó con un servicio de prueba.

15. Para confirmar: `CLAUDE.md` y `AGENTS.md`

- **Qué pasa:** en `src/` hay dos archivos, subidos en el commit "Avance 2", con una plantilla que pide instalar Laravel Boost (`composer require laravel/boost --dev`). Boost no está en `composer.json` y los archivos no afectan la ejecución de la aplicación.
- **Pregunta:** ¿se agregaron a propósito? Si no se usan, se pueden retirar; si se usan, conviene decidir si se instala Boost.
- **Verificación:** leído en el repositorio.